UNIDAD DE TRABAJO 8

TIPOS ABSTRACTOS DE DATOS (TAD)

–

Módulo de Programación

This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License.

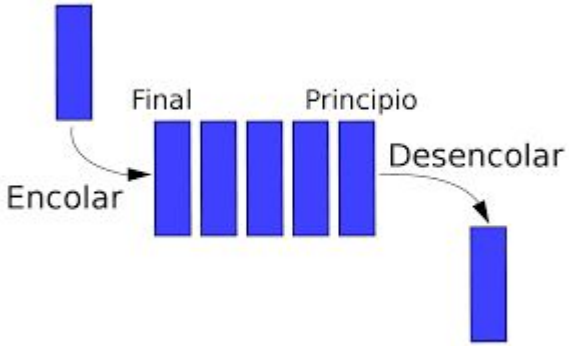
Autor: Fran Gómez

Índice de contenidos resumido

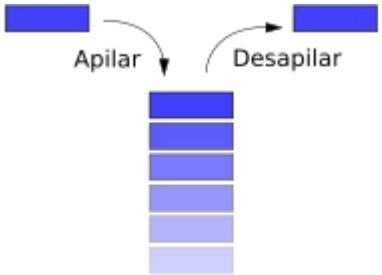
1. Introducción a los TAD
2. Colecciones (JCF)
   2.1. Collection
   2.2. List
      2.2.1. ArrayList
      2.2.2. LinkedList
   2.3. Set
      2.3.1. HashSet
      2.3.2. LinkedHashSet
   2.4. SortedSet
      2.4.1. TreeSet
   2.5. Map
      2.5.1. HashMap
      2.5.2. LinkedHashMap
   2.6. SortedMap
      2.6.1. TreeMap
3. Wrappers
4. Inmutables
5. Tipos genéricos
6. Comparable y Comparator
7. Iterator y ListIterator
8. Collections

Ejemplos

TAD Cola



TAD Pila



Wikipedia

TAD: Definición

Modelo definido para representar tipos abstractos como una pila o un conjunto ⟶ Creamos un nuevo tipo de dato más avanzado

La evolución ha sido:

1. Tipos primitivos
2. Tipos envoltorios (ej. Integer) y objetos (ej. String, arrays,)
3. Tipos definidos por nosotros (ej. clase Persona)
4. TAD (ej. conjunto)

Cada vez podemos **modelar problemas** más complejos, es decir, más **reales**.

TAD: Características

- Se especifican mediante una interfaz
- Se definen mediante la implementación de esas interfaces
- Contienen: datos + operaciones (POO)

TAD: Ejemplos de uso

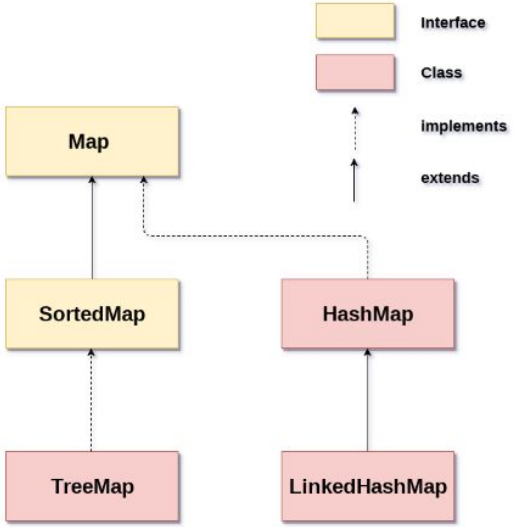
- **Listas:** una colección secuencial de elementos con operaciones como *agregar*, *borrar*, *recorrer*.
- **Pilas**: un colección de elementos que sigue la propiedad LIFO. Operaciones comunes incluyen *push*, *pop*, *top*.
- **Colas**: un colección de elementos que sigue la propiedad FIFO . Operaciones comunes incluyen *enQueue* o *agregarFinal*, *deQueue* o *retornarPrimero*.
- **Conjuntos**: implementación de conjuntos con sus operaciones básicas (unión, intersección y diferencia), operaciones de inserción, borrado, búsqueda...
- **Diccionarios**: son un conjunto de asociaciones de clave y valor. Sus operaciones comunes son *put* (colocar), *get* (obtener), recorrer, eliminar.
- **Grafos**: Implementación de grafos; una serie de vértices unidos mediante una serie de arcos o aristas.

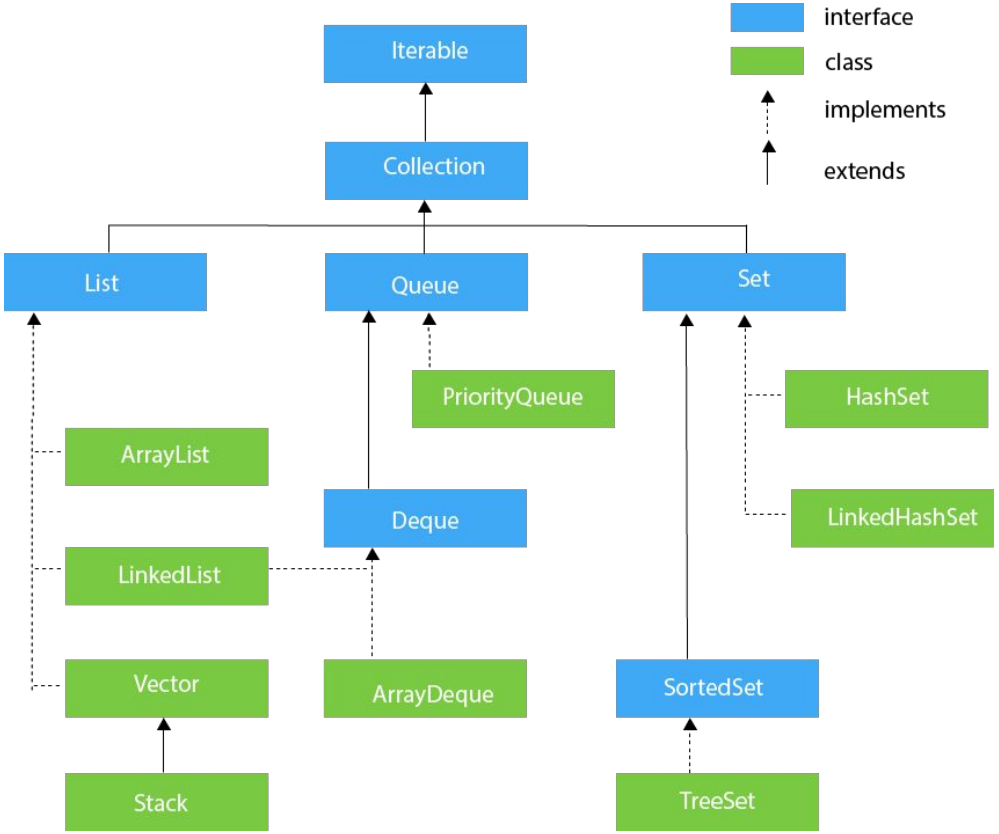
Fuente: wikipedia

Colecciones

Las colecciones son estructuras para guardar objetos.

Colecciones: Jerarquía





Paquete java.util.*

Java Collections Framework

Fuente: https://www.javatpoint.com/collections-in-java y https://www.javatpoint.com/java-map

Interfaz Collection

Representa una colección de elementos.

Se especializa en varios sub-interfaces según:

Interfaz	Ordenación	Duplicados	Nulos
Set	No (según implementación)	No	Sí (sólo uno)
SortedSet	Sí	No	Sí
List	Sí (orden de inserción)	Sí	Sí

Interfaz Map

Representa un diccionario de datos: clave - valor.

Se especializa en varios sub-interfaces según:

Interfaz	Ordenación	Duplicados	Nulos
Map	No garantiza un orden específico de los elementos.	No permite claves duplicadas, pero sí valores duplicados.	Depende de la implementación: HashMap permite una clave null, Hashtable no.
SortedMap	Ordena las claves de manera natural (Comparable) o mediante un Comparator personalizado.	Idem	TreeMap (implementación de SortedMap) no permite claves null, pero sí valores null.

Interfaz Collection

Provee de los métodos comunes a todos los sub-tipos de colecciones.

boolean **add**(E e);

boolean **remove**(Object o);

boolean **contains**(Object o);

int **size**();

boolean **isEmpty**();

void **clear**();

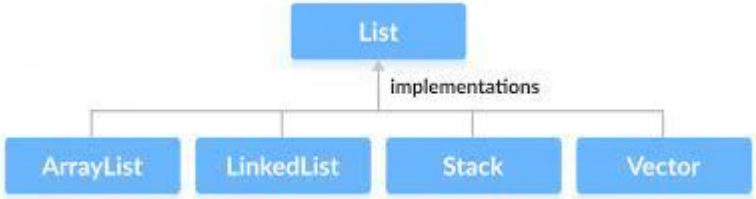
Iterator<E> **iterator**();

Podemos verlos todos en detalle en la documentación:

https://docs.oracle.com/javase/8/docs/api/java/util/Collection.html

Interfaz List

- Acceso a los elementos por su posición
- Permite repetidos
- Mantiene el orden de inserción



Fuente: Programiz

Interfaz	Ordenación	Duplicados	Nulos
Set	No	No	Depende
SortedSet	Sí	No	Sí
List	Sí	Sí	Sí

Interfaz List

Métodos de las listas:

void **add**(int index, E element);

E **get**(int index);

E **set**(int index, E element);

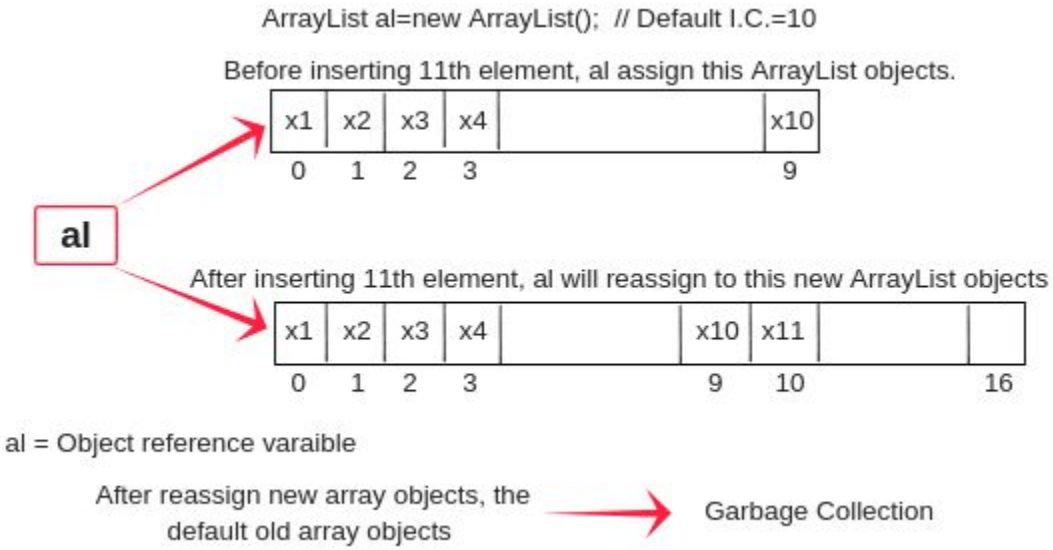
int **indexOf**(Object o);

E **remove**(int index);

ListIterator<E> **listIterator**();

Clase ArrayList

- Son Arrays dinámicos (tamaño se adapta al contenido)



Clase ArrayList

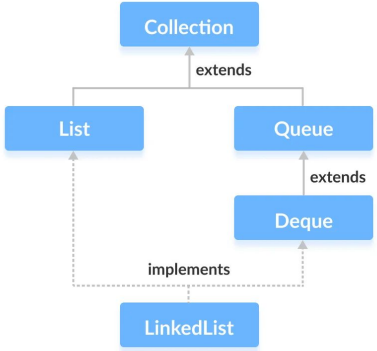
- Acceso aleatorio ⇒ Es la implementación más eficiente para las listas en operaciones de lectura
- Admite nulos
- Añade algunos métodos a los heredados de Collection. Son métodos que admiten posición.
- Recordar que el índice empieza en 0
- También pueden ser multidimensionales

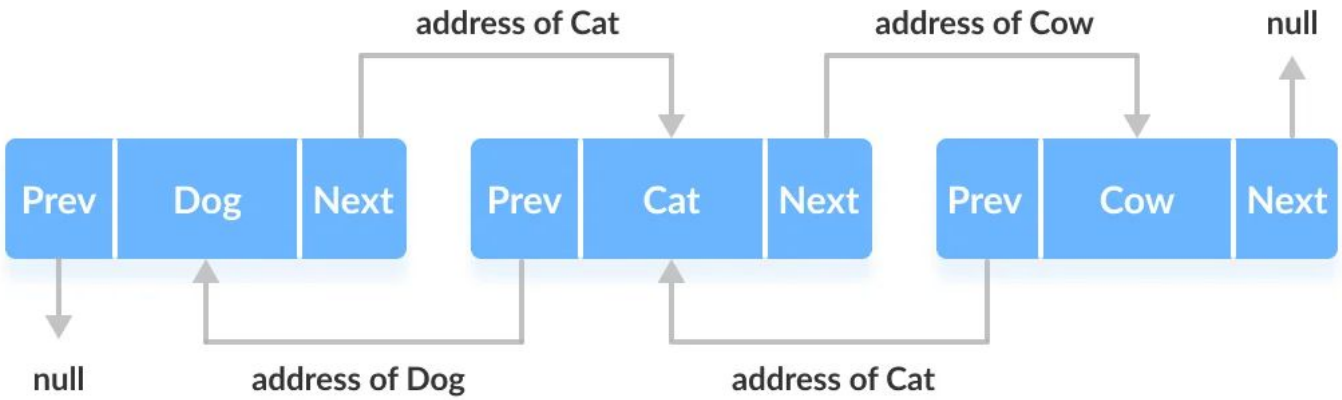
```
List<String> cars = new ArrayList<>();
cars.add("Volvo");
cars.add("BMW");
cars.add("Ford");
cars.add("Mazda");
System.out.println(cars);
```

```
[Volvo, BMW, Ford, Mazda]
```

Clase LinkedList

Lista enlazada: cada elemento tiene una referencia al anterior y al siguiente.





LinkedList Implementation in Java

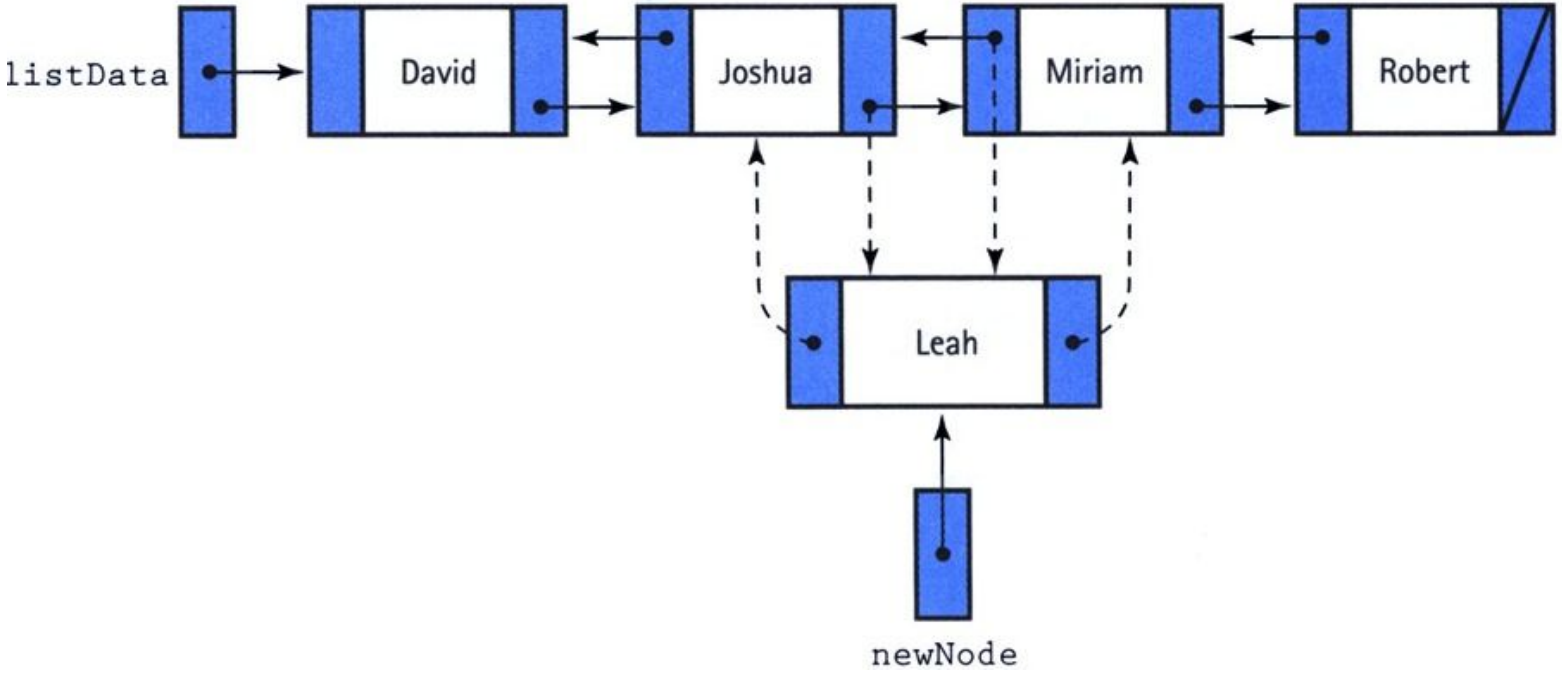
Fuente: setscholars.net

Clase LinkedList

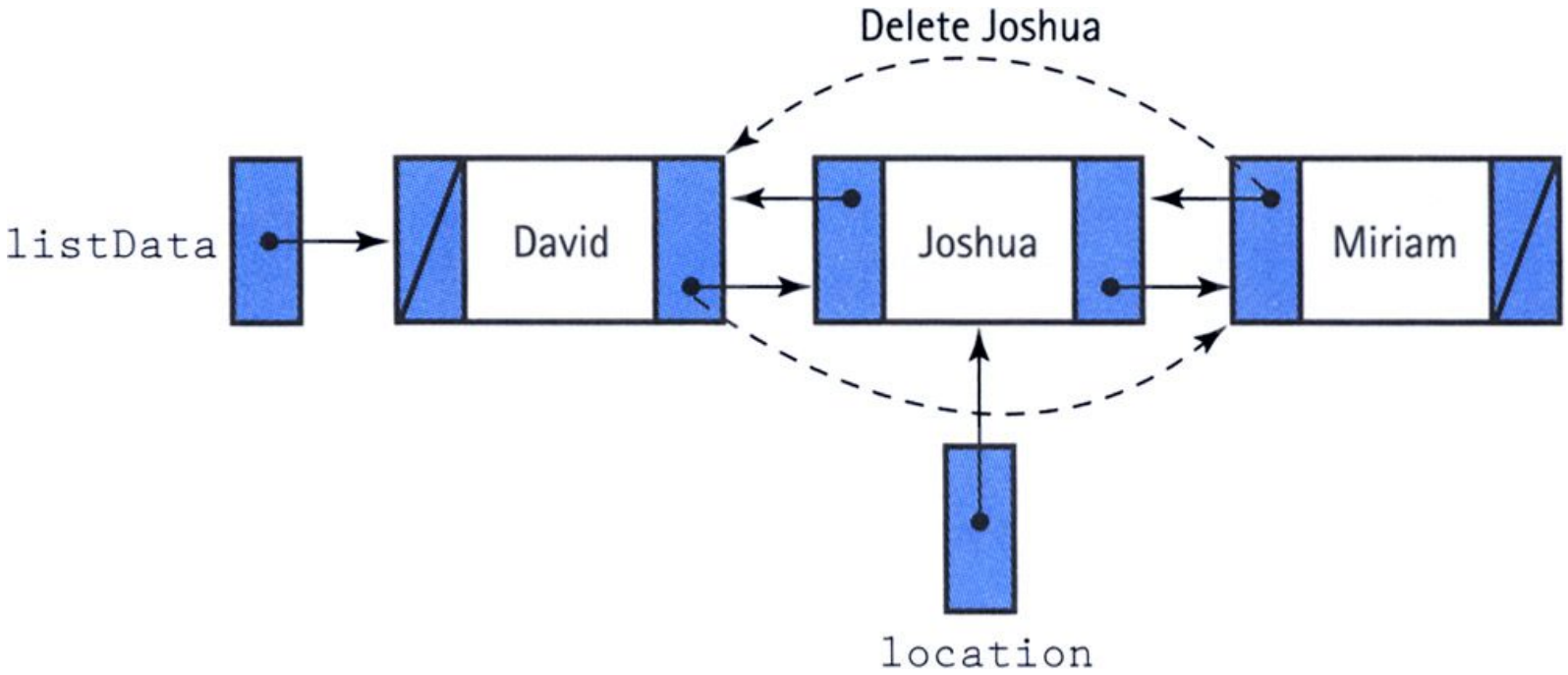
- Elementos no almacenados en memoria de forma contigua
- Acceso secuencial
  - Mal rendimiento para búsquedas
  - Buen rendimiento para inserciones y eliminaciones
- Permite null
- Añade más métodos para implementar colas y pilas
- Ya que tanto ArrayList como LinkedList implementan List, ambas tendrán todos los métodos de List en común, incluidos los de acceso aleatorio como get(i), aunque internamente cada una lo implemente de una forma

Clase LinkedList: inserción

Inserting into a doubly linked list



Clase LinkedList: eliminación



Ejemplo LinkedList

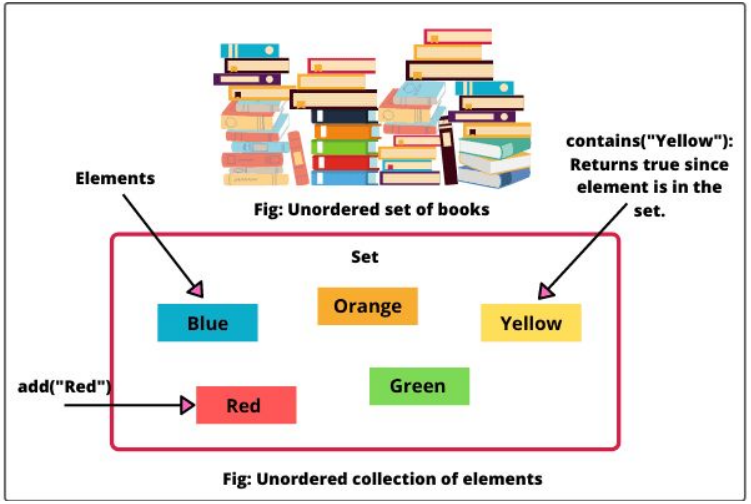

```
// Crear una LinkedList de Strings
List<String> lista = new LinkedList<>();

// INSERTAR elementos
lista.add("Manzana");          // Añadir al final
lista.addFirst("Banana");      // Añadir al principio
lista.addLast("Cereza");       // Añadir al final
lista.add(1, "Naranja");       // Insertar en posición específica
```

ArrayList vs LinkedList

Característica	`ArrayList`	`LinkedList`
Inserción/eliminación en medio o inicio	Lenta (requiere mover elementos)	Rápida (ajuste de enlaces)
Acceso por índice (`get(i)`)	Muy rápido (O(1))	Más lento (O(n))
Eficiencia general	Mejor para lectura frecuente	Mejor para inserciones/borrados
Implementa	`List`	`List` , `Deque` , `Queue`

Interfaz Set

- Implementa conjuntos de elementos
- Hereda de Collection
- Añade restricciones para que no pueda contener duplicados
- Según la implementación podrá estar o no ordenado



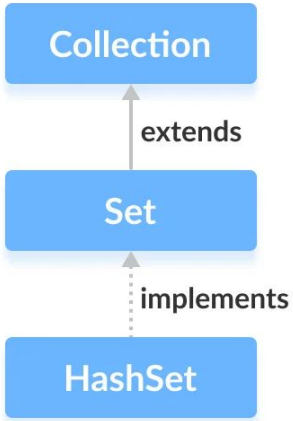
Sobreescribe el método:

boolean **add**(E e);

para no añadir si ya existe

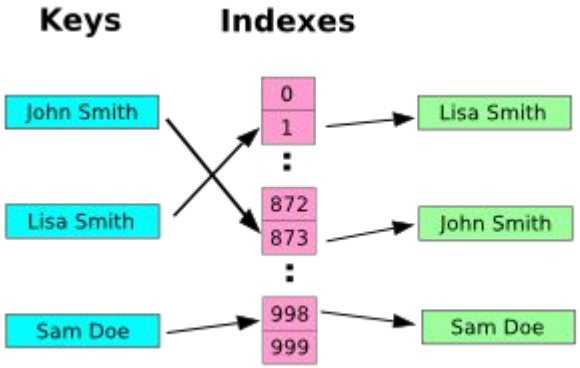
Fuente: https://www.scientecheasy.com/

Clase HashSet



Fuente: https://www.programiz.com/

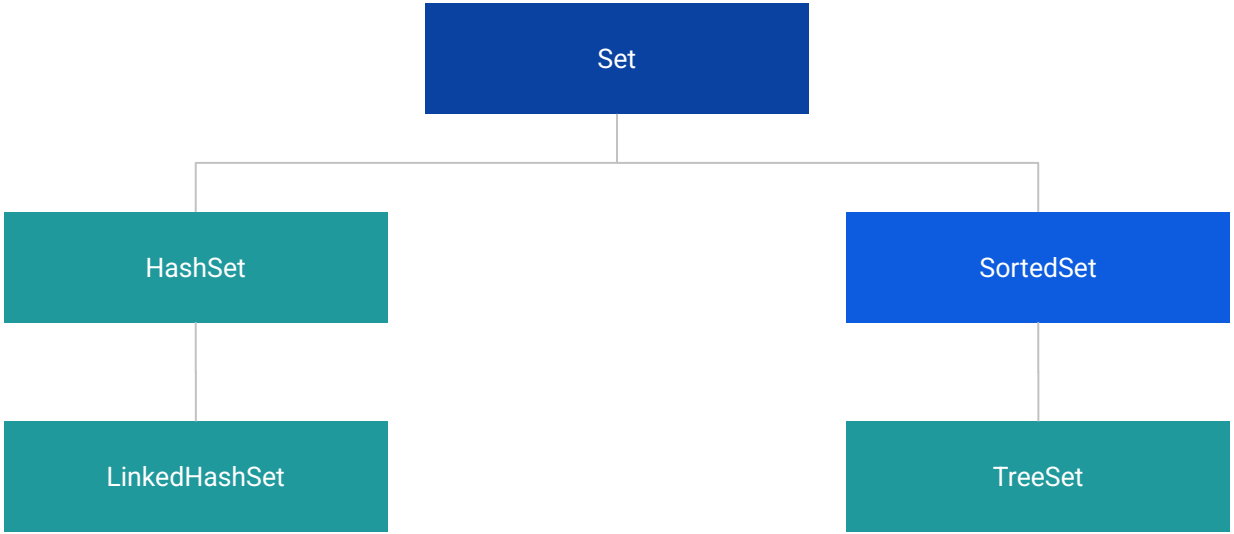
- Almacena los elementos en una tabla hash (usando la clase HashMap)
- Acceso rápido y aleatorio a los elementos



Clase HashSet: Ejemplo de código


```java
import java.util.HashSet;

class Main {
    public static void main(String[] args) {
        HashSet<Integer> numbers = new HashSet<>();
        numbers.add(1);
        numbers.add(2);
        numbers.add(3);
        numbers.add(4);
        System.out.println("HashSet1: " + numbers);

        HashSet<Integer> primeNumbers = new HashSet<>();
        primeNumbers.add(2);
        primeNumbers.add(3);
        System.out.println("HashSet2: " + primeNumbers);

        // Check if primeNumbers is a subset of numbers
        boolean result = numbers.containsAll(primeNumbers);
        System.out.println("Is HashSet2 is subset of HashSet1? " + result);
    }
}
```

Conjuntos



Implementaciones de los Conjuntos

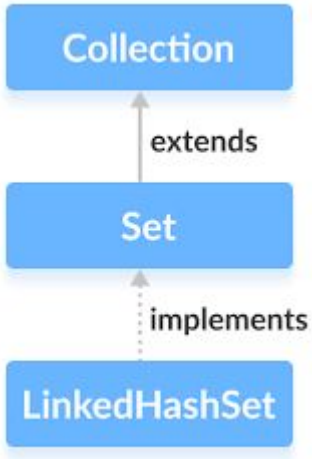
Tipo	¿Ordena los elementos?	¿Es rápido?	Uso ideal
HashSet	No, es un caos total.	El más rápido de todos (O(1)).	Cuando solo te importa que no haya repetidos.
LinkedHashSet	Por orden de llegada (como una cola).	Rápido.	Cuando quieres recordar quién llegó primero.
TreeSet	Orden alfabético o numérico.	Un poco más lento (O(log n)).	Cuando necesitas que la lista esté siempre ordenada.

Interfaces de los conjuntos

Característica	Set (Interfaz General)	SortedSet (Interfaz Especializada)
Duplicados	No permitidos.	No permitidos.
Orden	No garantizado (es un caos).	**Orden natural** (1, 2, 3... o A, B, C...).
Métodos extra	Básicos: add, remove, contains.	Avanzados: first(), last(), headSet().
Implementación	HashSet.	TreeSet.

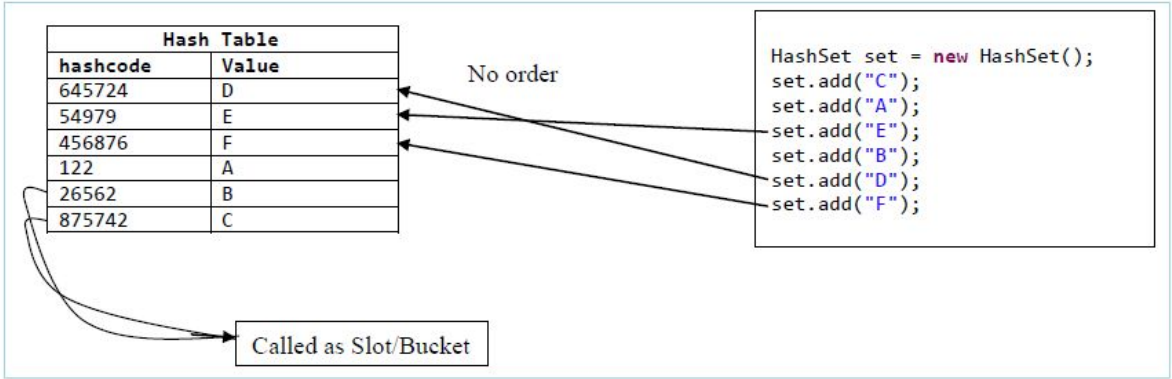
Clase LinkedHashSet

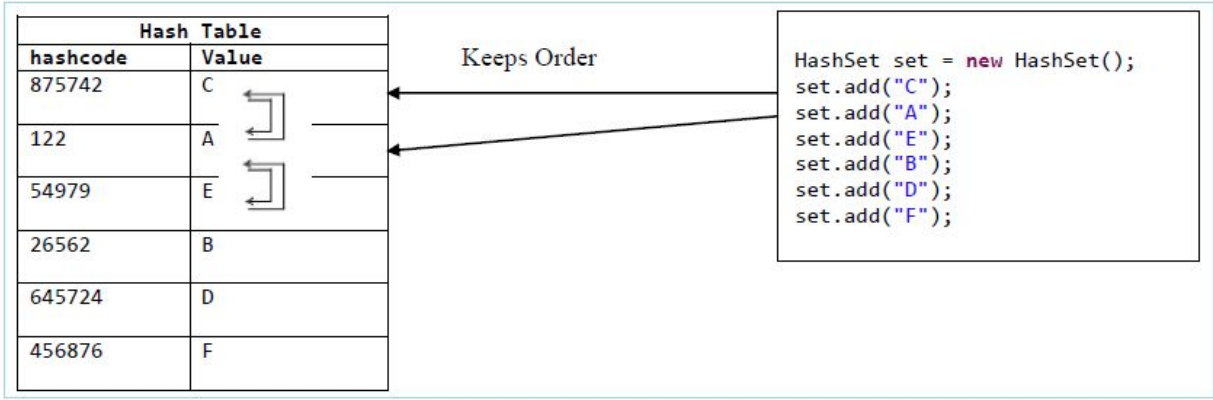
- Igual que HashSet, almacena elementos en una tabla hash, pero:
- Los elementos se ordenan por orden de inserción.




```
Set<String> linkedHashSet =new LinkedHashSet<>();
```

HashSet vs LinkedHashSet

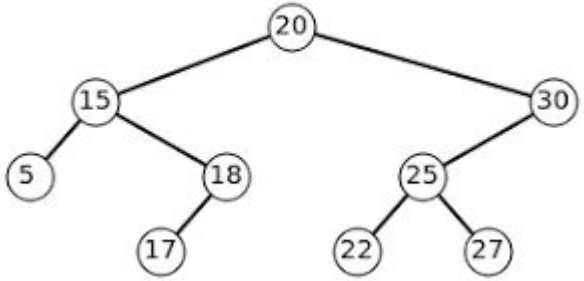




Clase TreeSet

- Utiliza estructura de árbol binario
- Elementos ordenados de forma ascendente según sus valores
- Es más lenta que HashSet, pero ofrece más funcionalidades
- No admite nulos

https://docs.oracle.com/javase/8/docs/api/java/util/TreeSet.html



Fuente: StackOverflow

```
TreeSet<Integer> treeSet = new TreeSet<>();
```

Ahora usamos la implementación en lugar de la interfaz para acceder a los métodos adicionales

Set: comparativa de implementaciones

Todas son implementaciones de la interfaz Set, con lo que tienen en común:

- No elementos repetidos

Pero cada uno además añade sus caracter propias:

¿Cuándo usar cada uno?

	HashSet	LinkedHashSet	TreeSet
Duplicados	No	No	No
Nulos		Sí	No
Orden	No	Inserción	Natural
Rendimiento	O(1)	O(1)	O(logn)

Objetos inmutables

Modificador *final*

- En una variable → no se puede reasignar (pero puede modificar el contenido si es un objeto mutable).
- En un método → no se puede sobreescribir.
- En una clase → no se puede extender.

```
final ArrayList<String> lista = new ArrayList<>();
lista.add("Hola"); // ✅ permitido ⇒ Objeto mutable
lista = new ArrayList<>(); // ❌ error de compilación
```

Objetos inmutables

Inmutable

- No se puede cambiar el contenido de ese objeto.
- Si se modifican se crea un nuevo objeto

Ejemplos:

- String
- Wrappers

```
String texto = "Hola";

texto.concat(" mundo"); // Intentamos modificar el contenido

System.out.println(texto); // Imprime: Hola

// Guardamos el resultado en una nueva variable
String nuevoTexto = texto.concat(" mundo");

System.out.println(nuevoTexto); // Imprime: Hola mundo
```

Tipos parametrizados o genéricos


```
List<String> conjunto = new ArrayList<>();
```

Tipo de datos que maneja la colección

Aquí siempre se deja vacío: **Diamond** operator

Indicar el **tipo** de elementos de una colección con **antelación**

Tipos parametrizados o genéricos

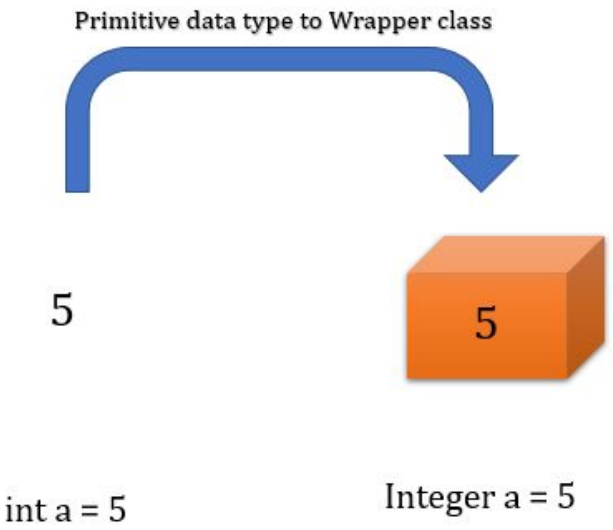

```
List<String> conjunto = new ArrayList<>();
```

¿Por qué son tan útiles?

- **Seguridad de Tipos (Type Safety):** El compilador detecta errores antes de que el programa se ejecute. Si intentas meter un gato en una lista de perros, el código ni siquiera compilará.

- **Eliminación de Casts:** Te olvidas de escribir `(String)` o `(Integer)` cada vez que sacas un elemento. El código queda mucho más limpio y legible.

Tipos envoltorio (Wrappers)



Java Goal

Primitive Data Types	Wrapper Classes
int	Integer
float	Float
double	Double
boolean	Boolean (Added from 1.5)
short	Short
byte	Byte
char	Character
long	Long

Dot Net Tutorial

Permiten tratar los tipos **primitivos** como **objetos**

Tipos envoltorio (Wrappers)


```
Integer siete = new Integer(7);
```

Está obsoleto (deprecated)

```
Integer siete = 7;
```

- Boxing

  Se **envuelve** el valor primitivo con un objeto

- Unboxing

  Se le **quita el envoltorio** para obtener de nuevo el valor primitivo

```
double x = 29.95;
Double y = x; // Boxing
x = y;        // Unboxing
```

Tipos envoltorio (Wrappers)

Métodos:

- **intValue()** // Unboxing
- **valueOf**() // Boxing
- parseInt() // Pasa de String a int
- toString() // Pasa a String

```
String str = "100";
Integer obj = Integer.valueOf(str);  // String → Integer (objeto)
int prim = obj.intValue();           // Integer → int (primitivo)
int directo = Integer.parseInt(str); // String → int (en un paso)[3][5].
```

Tipos envoltorio (Wrappers)


```
Set<int> hashSet = new HashSet<>();
```

Las colecciones no admiten tipos primitivos.

Hay que usar objetos → Wrappers

```
Set<Integer> hashSet = new HashSet<>();
```

```
hashSet.add(10); //Autoboxing
```

Iteradores

Una utilidad para recorrer colecciones

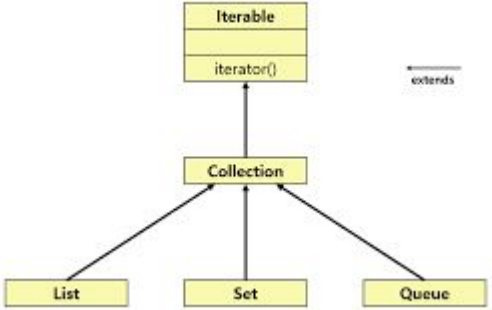
```
Set<String> conjunto = new HashSet<>();
conjunto.add("s1");
conjunto.add("s2");
conjunto.add("s3");
conjunto.add("s4");
conjunto.add("s5");
```

Ya conocíamos los bucles. Pero hay uno especialmente útil para las colecciones **for-each**

```
for (String s: conjunto) {
    System.out.println(s);
}
```

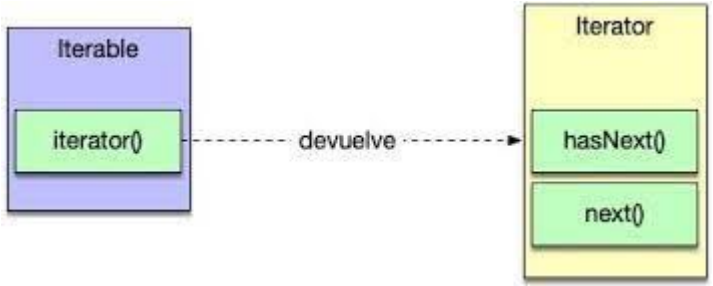
Ahora tenemos un TAD de Java creado para esta tarea: **Iterador**

```
Iterator<String> it = conjunto.iterator();
while (it.hasNext()) {
    System.out.print(it.next());
}
```



Iteradores

1. La interfaz Iterable provee a todas las colecciones la capacidad de obtener un iterador
2. La interfaz Iterator es el tipo de dato de este iterador, implementado en las colecciones. Esta interfaz tiene los métodos necesarios para realizar recorridos en las colecciones.



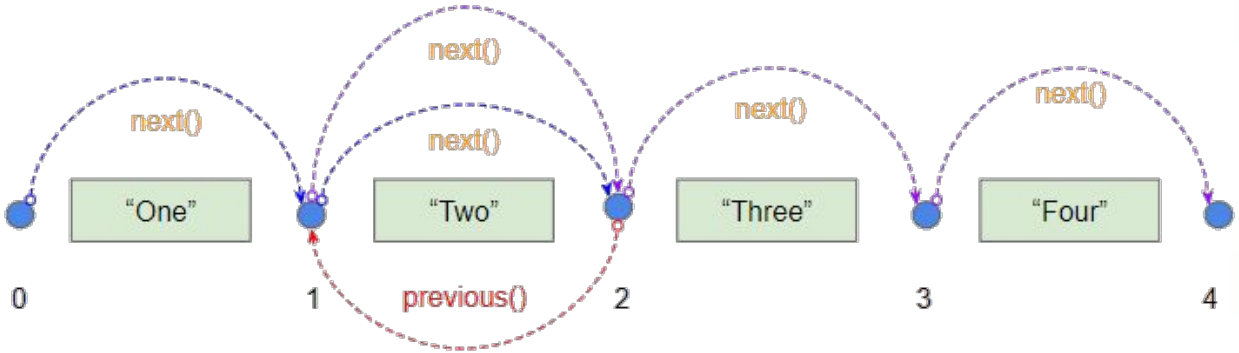
Fuente: https://www.arquitecturajava.com/

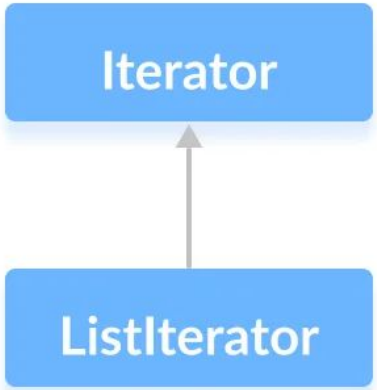
Interfaz ListIterator

Intentar eliminar elementos usando un bucle for o un bucle for-each no funcionaría correctamente porque la colección cambia de tamaño al mismo tiempo que el código intenta hacer un bucle. ⇒ Usar iteradores

Además de Iterator, las listas también tienen ListIterator:

- Permite recorrido en ambas direcciones





Comparadores: Interfaz Comparable

paquete: java.lang

- Da la capacidad a las clases que la implementan de establecer cuál es su orden natural
- String y envoltorios las tienen ya implementadas (sobreescrita)
- Tan sólo define un método: **compareTo**

```
package java.lang;

public interface Comparable<T> {

    int compareTo(T);

}
```

Debe implementarse para que devuelva:

- int > 0 ⇒ implícito > explícito
- int < 0 ⇒ implícito < explícito
- int == 0 ⇒ implícito = explícito

¡Conviene redefinir el método equals en concordancia!

Repasando

Si sobreescribes **equals()**, debes sobreescribir **hashCode()** de manera que:

si **a.equals(b)** es **true**, entonces **a.hashCode() == b.hashCode()** debe ser **true**.

Esto es crucial para el correcto funcionamiento de colecciones basadas en hash como HashMap o HashSet.

Si implementas Comparable, asegúrate de que la lógica de **compareTo()** sea consistente con la de **equals()**:

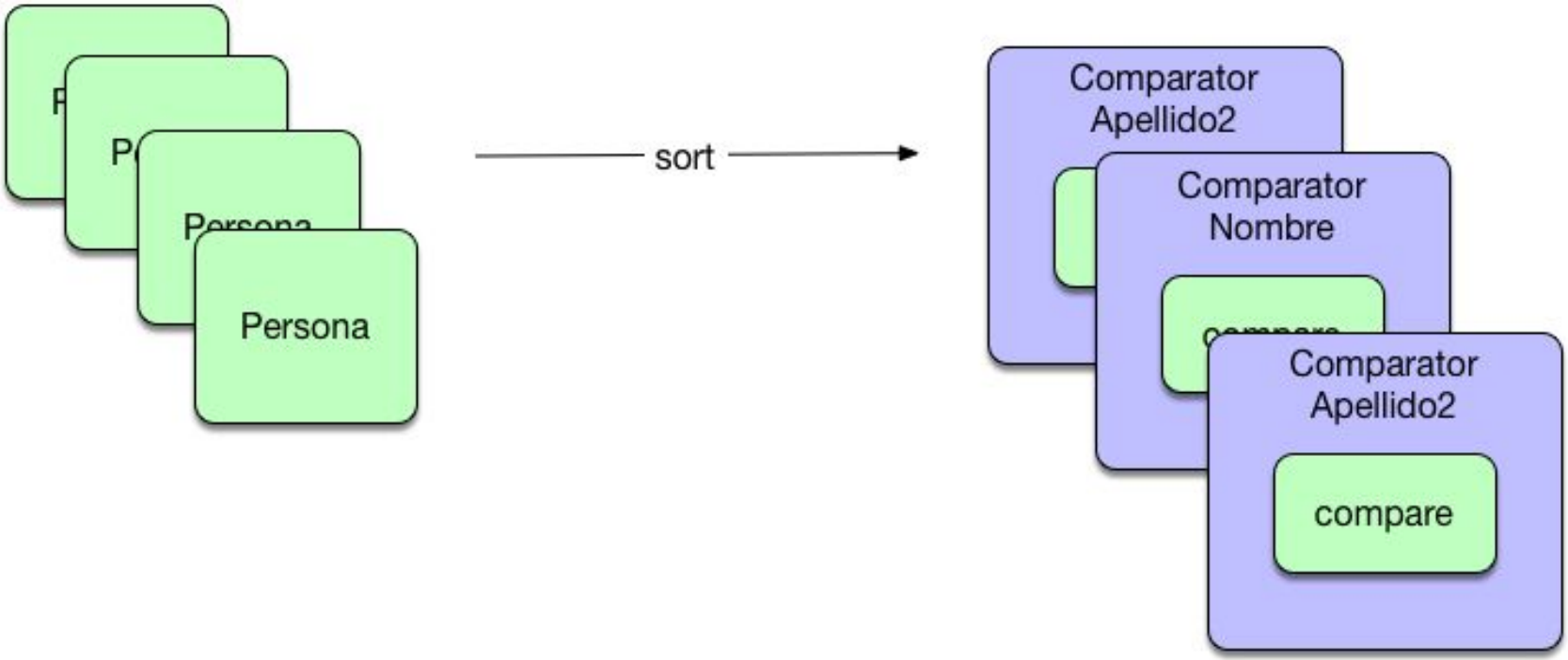
si **compareTo() == 0**, **equals() == true**.

Esto asegura que tus objetos se comporten correctamente en colecciones ordenadas como TreeSet o TreeMap.

Comparadores: Interfaz Comparator

paquete: java.util

Establece diversos **órdenes alternativos** para la clase



Comparadores: Interfaz Comparator

paquete: java.util

Establece diversos **órdenes alternativos** para la clase

- Se consigue creando una clase nueva por cada comparador alternativo que queramos hacer.
- Estos comparadores deben implementar Comparator, que contiene varios métodos para realizar comparaciones, pero sólo uno nos obliga a sobreescribir:

`int compare(Object o1, Object o2);`	Compara o1 y o2 devolviendo un entero en función del resultado (negativo, 0 o positivo). No hay parámetro implícito.

¡Conviene redefinir el método equals en concordancia!

Comparadores: Interfaz Comparator


```java
public class ComparadorPorEdad implements Comparator<Persona> {

    @Override
    public int compare(Persona p1, Persona p2) {

        if (p1.getEdad() == p2.getEdad())
            return 0;
        if (p1.getEdad() > p2.getEdad())
            return 1;
         else
            return -1;
    }
}
```

Comparadores: Interfaz Comparator


```
public static void main(String[] args) {
        Persona fran = new Persona("Fran",18);
        Persona ana = new Persona("Ana",19);
        System.out.println(fran.compareTo(ana));

        TreeSet<Persona> treeSet = new TreeSet<>();
        treeSet.add(fran);
        treeSet.add(ana);
        System.out.println(treeSet); → Ana, Fran

        TreeSet<Persona> treeSet2 = new TreeSet<>(new ComparadorPorEdad());
        treeSet2.add(fran);
        treeSet2.add(ana);
        System.out.println(treeSet2); → Fran, Ana
    }
```

Comparadores: Interfaz Comparator

Programación funcional al rescate:

```
Comparator<Persona> comparadorPersonaPorNombre = Comparator.comparing(Persona::getNombre);
```

¿Algún atajo?

```
Set<Persona> personas = new TreeSet<>(comparadorPersonaPorNombre);

Persona fran = new Persona("Fran",1);

Persona ana = new Persona("Ana",2);

personas.add(fran);

personas.add(ana);

System.out.println(personas);
```

Funciones lambda:

- Comparator.comparing(**p -> p.nombre**)

Method references

- Comparator.comparing(**Persona::getNombre**)

Interfaz SortedSet

Problema: La interfaz `Set` no declara métodos de ordenación como `first()`, `last()`, `headSet()`, etc. ⇒ Usamos **SortedSet**

Representa conjuntos ordenados

- Implementada por TreeSet
- Implementa Comparable

  Usa el método compareTo para ordenar

- Admite Comparator

  Podemos pasarle un Comparator

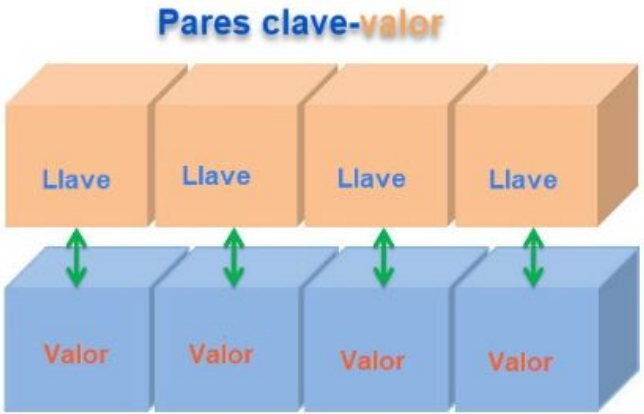
- No valores nulos
- Tiene menos métodos que TreeSet

```
SortedSet<String> abc = new TreeSet<>();
abc.add("B");
abc.add("C");
abc.add("A");
System.out.println(abc);        [A, B, C]

SortedSet<String> abc = new HashSet<>();
```

Nota: *Programar con **interfaces** mejor que con implementaciones*

Interfaz Map

Diccionario: tipo de dato que sirve para asociar **pares** de valores de la forma: **clave - valor**. Un Map en Java representa un diccionario.



Fuente: Guru99

Propiedades:

- Las claves no se repiten (una misma clave no puede estar asociada a distintos valores)
- Los valores sí pueden repetirse
- Un valor se obtiene a partir de su clave mediante acceso aleatorio

Interfaz Map

¿Cuándo usamos esta estructura de datos?

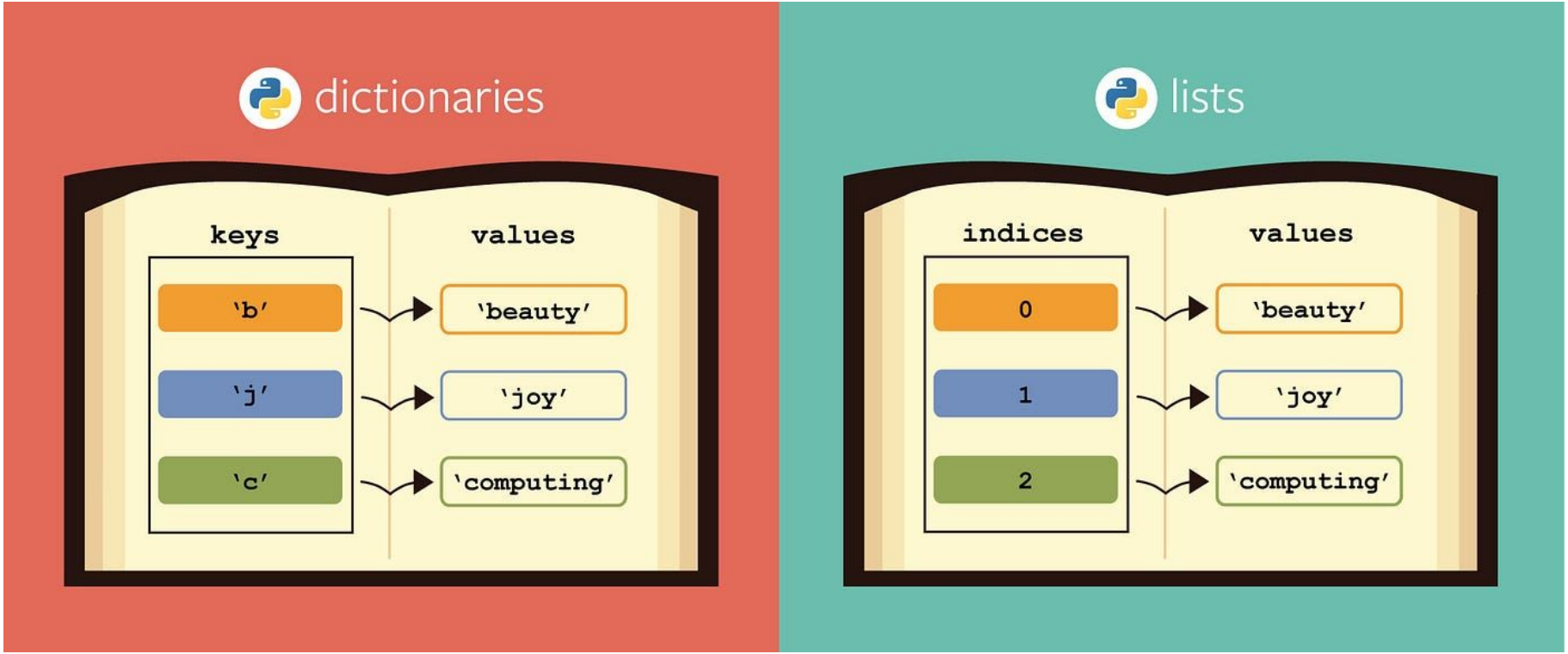
Almacenar valores asociados a claves para operar con ellos de forma eficiente. ¿Por ejemplo?

- Gestión de inventarios:
  - Clave: Código del producto
  - Valor: Cantidad disponible

- Configuraciones de aplicaciones:
  - Clave: Nombre del parámetro
  - Valor: Valor del parámetro

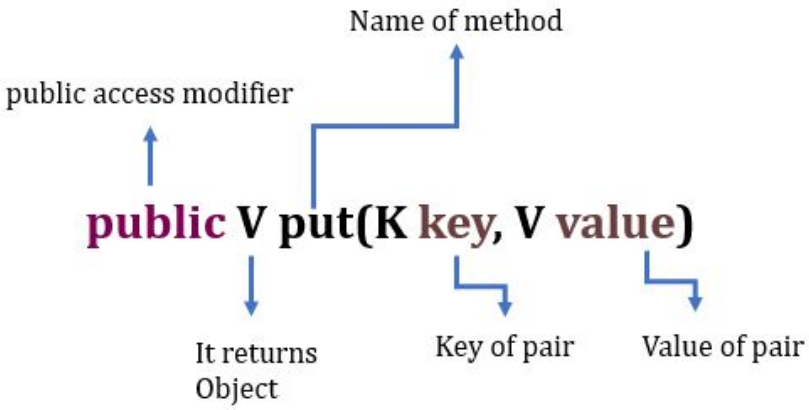
Clave (K)	Valor (V)
"ID_450"	35
"ID_122"	12
"ID_899"	35 (El valor puede repetirse)

Interfaz Map: Ejemplos



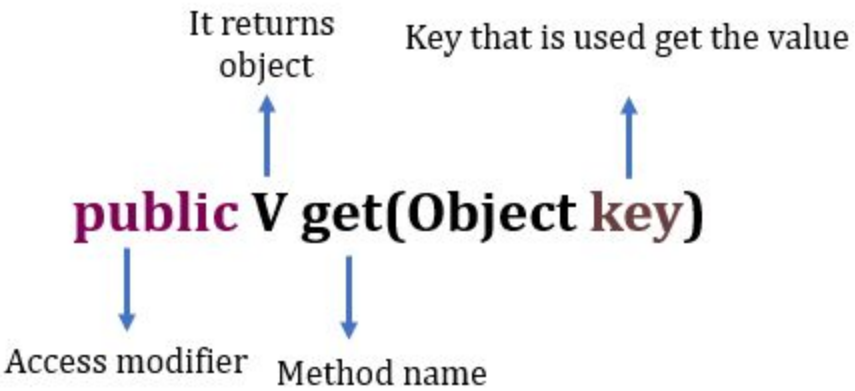
Interfaz Map: métodos

- **Adición**



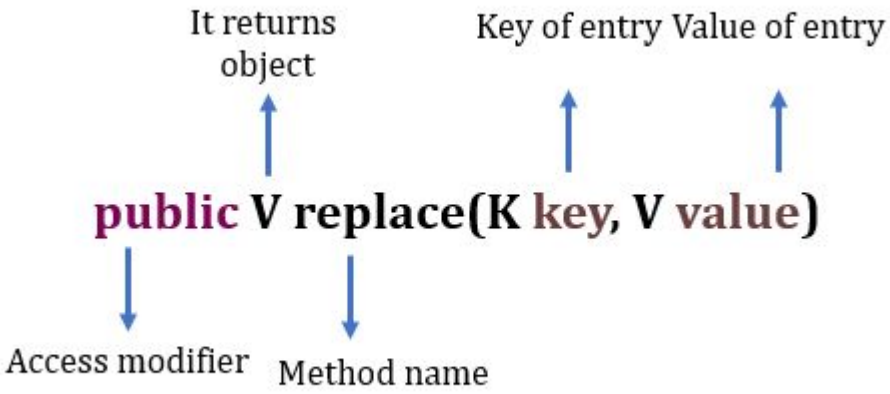
Interfaz Map: métodos

- Adición
- **Lectura**



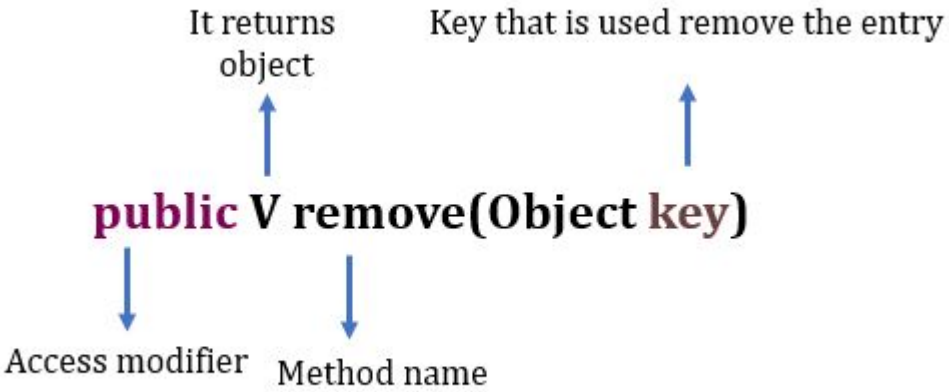
Interfaz Map: métodos

- Adición
- Lectura
- **Modificación**



Interfaz Map: métodos

- Adición
- Lectura
- Modificación
- **Eliminación**

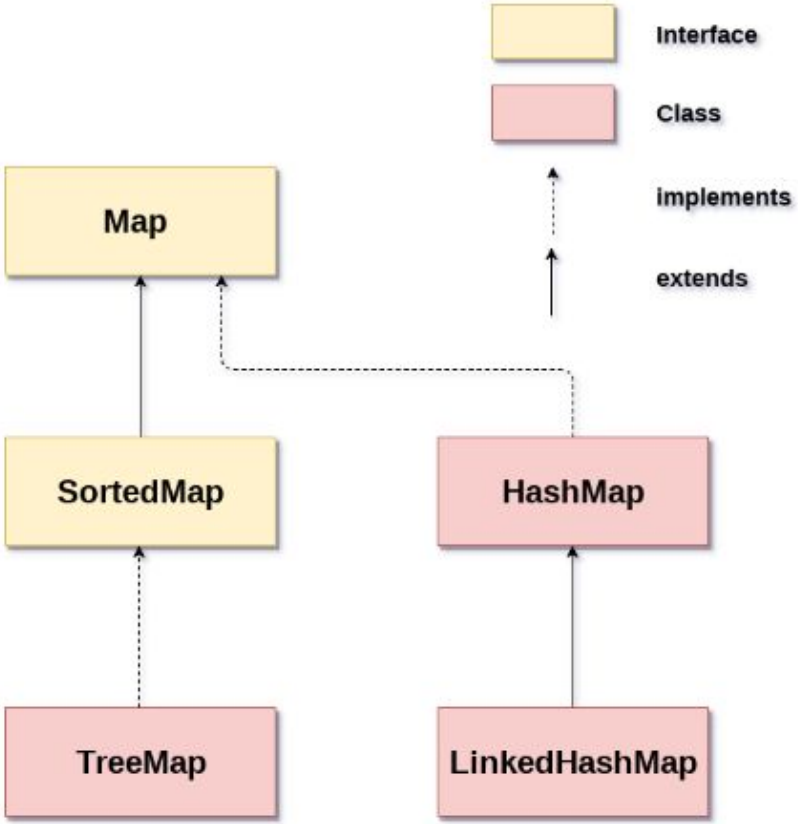


Interfaz Map: métodos

Otros métodos

- containsKey(Object key): Verifica si existe una clave.
- containsValue(Object value): Verifica si existe un valor.

Implementaciones de Map



Implementaciones de Map

Implementación	Características principales
HashMap	No garantiza orden; permite claves y valores nulos; operaciones rápidas (O(1)).
TreeMap	Ordena las claves de forma natural o según un comparador personalizado.
LinkedHashMap	Mantiene el orden de inserción.

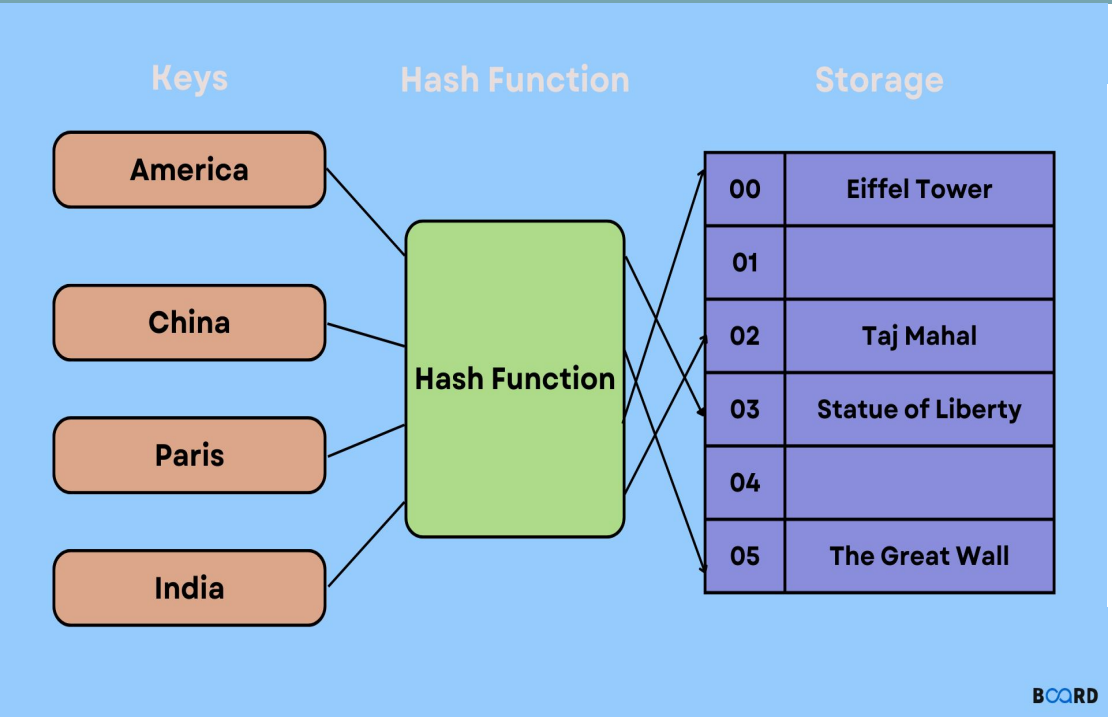
Clase HashMap

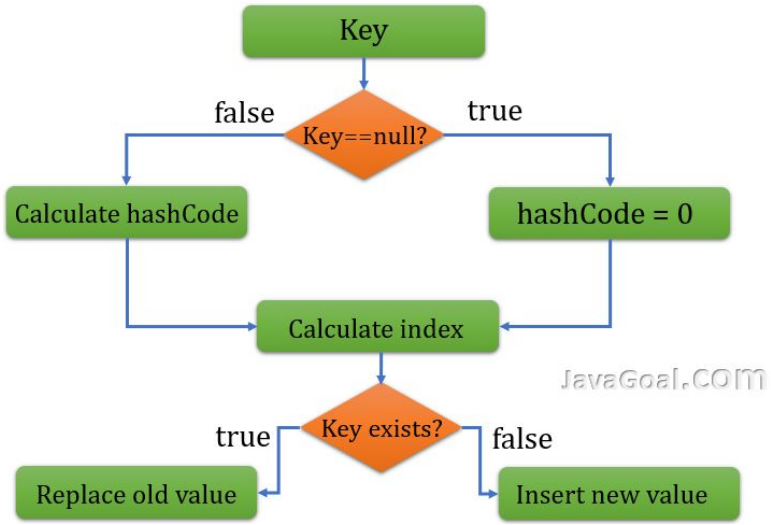
Características:

- Utiliza una tabla hash
- Buen rendimiento
- No garantiza ningún orden
- Sintaxis. Ejemplo:

```
HashMap<Integer, String> map = new HashMap<>();

map.put(1,"1");
```

Clase HashMap

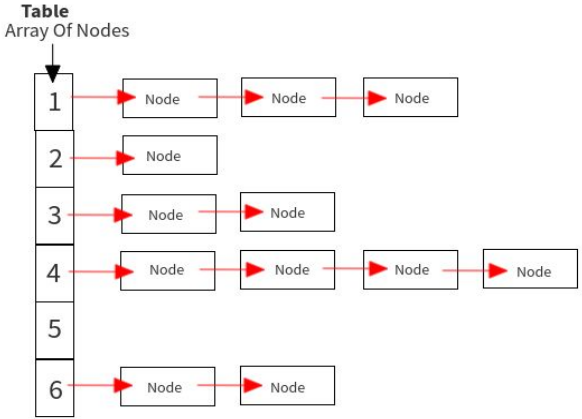




Clase LinkedHashMap

Implementa Map mediante una combinación de lista enlazada y tabla hash

```
public class LinkedHashMap<K,V> extends HashMap<K,V> implements Map<K,V>
```



Características:

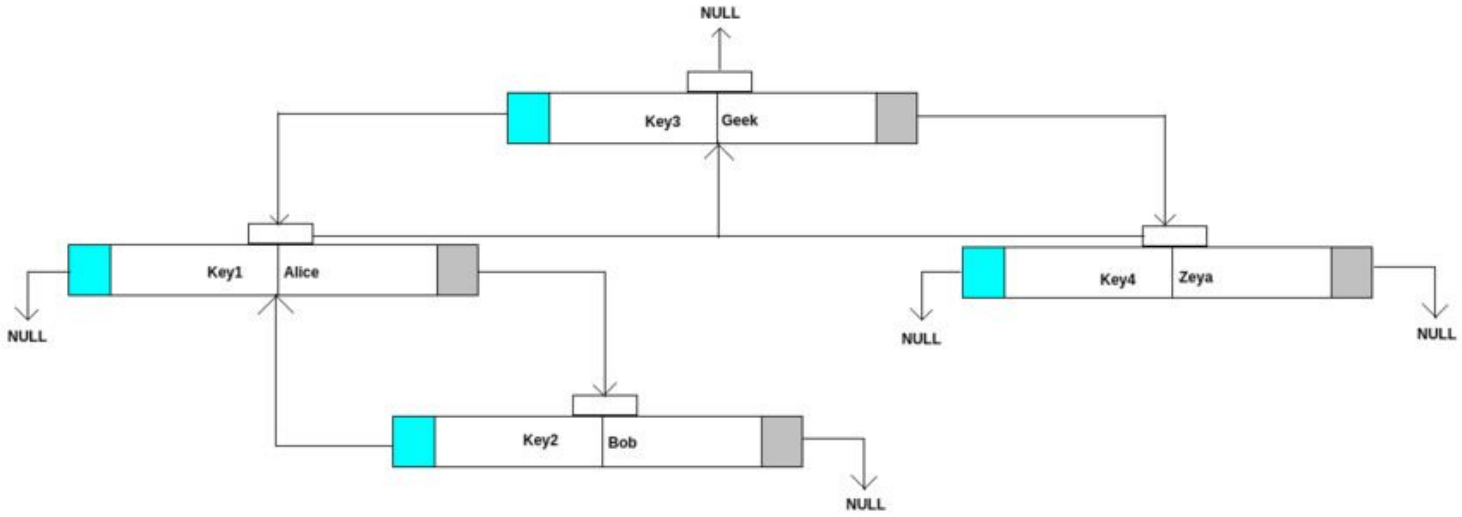
- Mantiene orden de inserción
- Sintaxis:

```
LinkedHashMap<K, V> lhm = new LinkedHashMap<K, V>();
```

Clase TreeMap

Características:

- Respeta orden natural de las claves (estas deben implementar Comparable)
- Más lento que HashMap
- No admite claves null
- Añade algunos métodos más



Interfaz Map

Representa un diccionario de datos: clave - valor.

Se especializa en varios sub-interfaces según:

Interfaz	Ordenación	Duplicados	Nulos
Map	No garantiza un orden específico de los elementos.	No permite claves duplicadas, pero sí valores duplicados.	Depende de la implementación: HashMap permite una clave null, Hashtable no.
SortedMap	Ordena las claves de manera natural (Comparable) o mediante un Comparator personalizado.	Idem	TreeMap (implementación de SortedMap) no permite claves null, pero sí valores null.

Ejemplo


```
public class EjemploMap {
    public static void main(String[] args) {
        // Crear un HashMap
        Map<String, String> m= new HashMap<>();
        // Insertar pares clave-valor
        m.put("123", "Juan Pérez");
        m.put("456", "Ana Gómez");
        m.put("789", "Luis Rodríguez");
        // Acceder a un valor por su clave
        System.out.println("Valor asociado a la clave '123': " + m.get("123"));
```

¿Cómo recorrer un Map?

- Usando el conjunto de las claves: método **keySet**()

```
// Recorrer el mapa con for-each sobre keySet
for (String clave : m.keySet()) {
        System.out.println("Clave: " + clave + ", Valor: " + m.get(clave));
}
```

- Usando el conjunto de los valores: método **entrySet**()

```
for (Map.Entry<String, String> entrada: m.entrySet()) {
                System.out.println("Clave: " + entrada.getKey() + ", Valor: " +
entrada.getValue());
            }
```

Clase de utilidades: Collections

Diferencia entre `Collection` (interfaz) y `Collections` (clase utilitaria).

Importación: `import java.util.Collections;`

Ofrece operaciones útiles para todas las colecciones

Contiene métodos estáticos.

```
Collections.sort(nombres);
```

Métodos útiles de Collections

- `sort(List<T>)` y `sort(List<T>, Comparator<? super T>)`
- `reverse(List<?>)`
- `shuffle(List<?>)`
- `swap(List<?>, int, int)`
- `max(Collection<?>)` / `min(Collection<?>)`
- `frequency(Collection<?>, Object)`
- `fill(List<? super T>, T)`
- `copy(List<? super T>, List<? extends T>)`

https://docs.oracle.com/javase/8/docs/api/java/util/Collections.html

Casos de uso interesantes

Ordenar una lista de nombres

```
List<String> nombres = new ArrayList<>(List.of("Lucía", "Carlos", "Ana", "Pedro"));
Collections.sort(nombres);
System.out.println(nombres); // [Ana, Carlos, Lucía, Pedro]
```

Barajar una lista de números

```
List<Integer> numeros = new ArrayList<>(List.of(1, 2, 3, 4, 5));
Collections.shuffle(numeros);
System.out.println(numeros); // orden aleatorio
```

Encontrar el valor máximo y mínimo

```
List<Integer> notas = List.of(5, 7, 9, 4, 6);
System.out.println("Nota más alta: " + Collections.max(notas));
System.out.println("Nota más baja: " + Collections.min(notas));
```

Contar la frecuencia de una palabra

```
List<String> palabras = List.of("sol", "luna", "sol", "estrella");
System.out.println("Veces que aparece 'sol': " + Collections.frequency(palabras, "sol"));
```

Fin

1 Go to wooclap.com

2 Enter the event code in the top banner

Event code
PROGUT5

Enable answers by SMS